

LAPORAN TUGAS BESAR 2
IF3270 Pembelajaran Mesin

**Convolutional Neural Network dan Recurrent Neural
Network**



Disusun Oleh Kelompok OpenUpthesky:

Sebastian Enrico Nathanael 13523134

Frederiko Eldad Mugiyono 13523147

Benedictus Nelson 13523150

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
PENDAHULUAN.....	3
1.1 Deskripsi Persoalan.....	3
1.1.1 CNN untuk Image Classification.....	3
1.1.2 RNN/LSTM untuk Image Captioning.....	3
BAB II	
PENJELASAN IMPLEMENTASI.....	4
2.1 Implementasi Kelas.....	4
2.1.1 Modul Shared.....	4
2.1.2 Modul CNN.....	6
2.1.3 Modul RNN/LSTM.....	9
2.1.4 Training & Evaluation Pipeline.....	13
2.2.1 Forward Propagation CNN.....	16
2.2.3 Forward Propagation LSTM.....	17
2.3 Pipeline Image Captioning (Pre-Inject).....	18
2.4 Proses Training Model.....	19
BAB III	
PERCOBAAN DAN ANALISIS.....	23
3.1 CNN.....	23
3.1.1.....	23
3.1.2.....	23
3.1.3.....	23
3.1.4.....	24
3.1.5.....	24
3.1.6.....	24
3.2 RNN.....	25
3.2.1.....	25
3.2.2.....	25
3.2.3.....	26
3.2.4.....	26
3.2.5.....	26
3.2.6.....	27
3.2.7.....	27
BAB IV	
KESIMPULAN DAN SARAN.....	29
4.1 Kesimpulan.....	29
4.2 Saran.....	29

Pembagian Tugas.....	30
Lampiran.....	32
Referensi.....	34

BAB I

PENDAHULUAN

1.1 Deskripsi Persoalan

Pada Tugas Besar 2 mata kuliah IF3270 Pembelajaran Mesin ini, ditugaskan untuk mengimplementasi dua arsitektur *deep learning* fundamental yaitu *Convolutional Neural Network* (CNN) dan *Recurrent Neural Network* (RNN/LSTM) beserta seluruh tahap forward propagation nya secara "*from scratch*" menggunakan NumPy. Tujuannya adalah agar peserta kuliah memahami matematika di balik setiap layer (alih-alih hanya memanggil API), serta mampu mengintegrasikannya menjadi *pipeline image captioning end-to-end* sesuai *Show and Tell* (Vinyals et al., 2015).

1.1.1 CNN untuk Image Classification

Dataset *Intel Image Classification* (~25.000 gambar, 6 kelas: *buildings, forest, glacier, mountain, sea, street*). Tugas:

- Membangun pipeline preprocessing image (PIL + NumPy, tanpa Keras preprocessing).
- Melatih model CNN baseline berbasis Conv2D shared-parameter di Keras dengan *sweep* hyperparameter (jumlah layer konvolusi, jumlah filter, ukuran filter, jenis pooling), total 16 arsitektur.
- Reimplementasi forward pass *from-scratch* untuk Conv2D, LocallyConnected2D, MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D, GlobalMaxPooling2D, Flatten, Dense, dan fungsi aktivasi (ReLU, Softmax, dst.) menggunakan NumPy.
- Membandingkan hasil prediksi Keras vs implementasi from-scratch menggunakan macro F1-score; memeriksa pengaruh parameter sharing dengan mengganti seluruh Conv2D menjadi LocallyConnected2D.

1.1.2 RNN/LSTM untuk Image Captioning

Dataset Flickr8k (8.092 gambar, 5 caption per gambar) dengan split train/val/test 6.000/1.000/1.000. Arsitektur: *pre-inject* (Tanti et al., 2017) yang menempatkan fitur CNN sebagai token x_{-1} sebelum embedding token $\langle \text{start} \rangle$. Tugas:

- Mengekstrak feature vector seluruh gambar Flickr8k menggunakan InceptionV3 frozen (bobot ImageNet, tanpa top layer).
- Membangun pipeline preprocessing caption (lowercase, hapus tanda baca, wrap $\langle \text{start} \rangle / \langle \text{end} \rangle$, vocabulary min_freq=2, padding).
- Melatih dua decoder (SimpleRNN dan LSTM) dengan *sweep*: 3 jumlah layer $(1, 2, 3) \times 2$ ukuran hidden state (128, 512) = 12 variasi total.
- Reimplementasi *from-scratch*: Embedding, SimpleRNN, LSTM, Dense projection, Dense output, serta integrasinya menjadi ImageCaptionerScratch.
- Evaluasi BLEU-4 dan METEOR pada test set; perbandingan RNN vs LSTM, Keras vs from-scratch, serta pengaruh max caption length.

BAB II

PENJELASAN IMPLEMENTASI

2.1 Implementasi Kelas

Implementasi disusun dalam paket `src/` yang terbagi menjadi tiga sub-paket utama: `src/shared/` berisi utilitas bersama untuk pemuatan gambar, fungsi aktivasi, dan layer Dense; `src/cnn/` berisi seluruh komponen CNN baik dari sisi Keras maupun *from-scratch*; dan `src/rnn_lstm/` berisi seluruh komponen decoder image captioning. Setiap layer *from-scratch* menyediakan classmethod `from_keras_layer()` yang membaca `get_weights()` dari layer Keras dan menyimpannya pada atribut internal, sehingga pengguna cukup memanggil `ScratchModel.load_from_keras(keras_model)` untuk memperoleh model *from-scratch* yang ekuivalen secara numerik dengan model Keras hasil training.

2.1.1 Modul Shared

Modul `src/shared/` menyediakan utilitas yang dipakai bersama oleh CNN dan RNN/LSTM. Modul ini terdiri dari tiga file utama, yaitu `image_utils.py` (pemuatan dan preprocessing gambar), `activations.py` (kumpulan fungsi aktivasi NumPy-only), dan `dense_layer.py` (kelas `DenseLayer` yang dipakai pada CNN sebagai output layer maupun RNN/LSTM sebagai *projection* dan *output layer*).

Method/Atribut	Deskripsi
<code>load_image(path, target_size)</code>	Membuka gambar dengan PIL, konversi ke RGB, resize ke <code>target_size</code> , lalu normalisasi piksel ke <code>[0,1]</code> . Mengembalikan <code>np.ndarray</code> shape (H, W, 3) dtype float32.
<code>load_batch(paths, target_size)</code>	Melakukan loop <code>load_image</code> kemudian <code>np.stack</code> ke shape (N, H, W, C).
<code>extract_and_save_features(paths, encoder, out_path, ...)</code>	<i>Forward</i> batch gambar melalui Keras encoder frozen (mis. InceptionV3), simpan hasil ke file .npy dan mapping {filename: index} ke file JSON. Mendukung skip jika file output sudah ada.
<code>relu / softmax / sigmoid / tanh / leaky_relu / elu / linear</code>	Implementasi NumPy-only seluruh fungsi aktivasi yang dipakai. Softmax menggunakan <i>log-sum-exp</i> stabilisasi ($x - x.\max(\dots)$) sebelum eksponensiasi untuk mencegah <i>overflow</i> .
<code>get_activation(name)</code>	Dispatcher string ke fungsi. Dipakai oleh <code>DenseLayer</code> dan <code>Conv2DLayer</code> ketika me-load aktivasi dari konfigurasi Keras.

DenseLayer	Kelas representasi layer <i>fully-connected</i> . Atribut: weights shape (in_dim, out_dim), biases shape (1, out_dim), activation_fn (callable). <i>Method</i> forward(x) menghitung $z = x @ \text{weights} + \text{biases}$, lalu menerapkan activation_fn. <i>Method</i> from_keras_layer(layer) menyalin bobot dan bias dari layer Keras.
------------	--

activations.softmax & sigmoid

```
def softmax(x):
    shifted = x - np.max(x, axis=-1, keepdims=True)
    exp_x = np.exp(shifted)
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

def sigmoid(x):
    x_clipped = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x_clipped))
```

DenseLayer.forward & from_keras_layer

```
def forward(self, x):
    z = np.dot(x, self.weights) + self.biases
    return self.activation_fn(z)

@classmethod
def from_keras_layer(cls, keras_layer):
    kernel, bias = keras_layer.get_weights()
    act_name = keras_layer.activation.__name__
    layer = cls(kernel.shape[0], kernel.shape[1],
activation=act_name)
    layer.weights = kernel.copy()
    layer.biases = bias.reshape(1, -1).copy()
    return layer
```

2.1.2 Modul CNN

Modul `src/cnn/` berisi seluruh komponen CNN. File `model.py` dan `train.py` membangun dan melatih CNN Keras (Conv2D + Pooling + GlobalAveragePooling2D + Dense), sedangkan layer *from-scratch* dipecah menjadi file per kelas (`conv2d.py`, `locally_connected.py`, `pooling.py`, `flatten.py`). File `layers.py` berfungsi sebagai *shim re-export* untuk menjaga kompatibilitas path import yang dispesifikasikan dalam soal.

Method/Atribut	Deskripsi
<code>Conv2DLayer</code>	Layer konvolusi 2D dengan <i>parameter sharing</i> . Setiap posisi spasial menggunakan kernel yang sama, menghasilkan output $y[i,j,k]$ melalui <i>sliding window</i> lalu diikuti fungsi aktivasi.
<code>LocallyConnected2DLayer</code>	Layer konvolusi tanpa <i>parameter sharing</i> , setiap posisi spasial memiliki kernel sendiri. Atribut: kernel shape ($out_rows * out_cols, kH * kW * C_in, C_out$), bias shape ($out_rows * out_cols, C_out$). <i>Method</i> <code>as_locally_connected()</code> pada <code>CNNScratchModel</code> menukar tiap <code>Conv2DLayer</code> dengan <code>LocallyConnected2DLayer</code> untuk eksperimen shared vs non-shared.
<code>MaxPooling2DLayer</code> / <code>AveragePooling2DLayer</code>	Pooling spasial dengan <code>pool_size</code> dan <code>strides</code> . <i>Forward</i> melakukan <i>sliding window</i> dan <code>np.max</code> / <code>np.mean</code> per channel pada axis spasial.
<code>GlobalAveragePooling2DLayer</code> / <code>GlobalMaxPooling2DLayer</code>	Pooling global, mereduksi tensor (H, W, C) menjadi (C,) menggunakan <code>np.mean</code> atau <code>np.max</code> pada axis (0,1).
<code>FlattenLayer</code>	Reshape (N, H, W, C) $\rightarrow$ (N, H*W*C) dalam urutan <i>row-major</i> (order C), konsisten dengan <code>tf.keras.layers.Flatten()</code> .
<code>CNNScratchModel</code>	Container model <i>from-scratch</i> . Atribut: <code>layers</code> (list layer scratch). <i>Method</i> <code>load_from_keras(keras_model)</code> memuat bobot dari tiap layer Keras via <code>from_keras_layer()</code> . <i>Method</i> <code>forward(x)</code> menjalankan semua layer secara sekuensial. <i>Method</i> <code>as_locally_connected()</code> mengganti seluruh <code>Conv2D</code> dengan <code>LocallyConnected2D</code> .
<code>build_cnn_model(config)</code> [<code>model.py</code>]	Membangun <code>keras.Model</code> fungsional sesuai <code>CNNConfig</code> (<code>num_conv_layers</code> , <code>filters_per_layer</code> , <code>kernel_size</code> , <code>pooling_type</code>). Arsitektur: $[Conv2D(filter) \rightarrow ReLU \rightarrow Pool] \times N$, lalu $GlobalAveragePooling2D \rightarrow Dense(128, ReLU) \rightarrow Dense(6, Softmax)$.
<code>CNNConfig</code> + <code>train_grid</code>	Grid 16 variant: 2 jumlah layer $\times$ 2 kombinasi filter $\times$ 2

+ hyperparameter_grid [train.py]	ukuran kernel $\times$ 2 jenis pooling. train_grid() melatih semua variant dan menyimpan bobot, history, serta config ke models/cnn/.
grad_cam(...) [gradcam.py, BONUS]	Menghitung gradien logits target class terhadap aktivasi <i>convolution</i> terakhir, melakukan <i>global-average gradient</i> , <i>weighted sum</i> aktivasi, lalu ReLU dan <i>resize</i> ke ukuran <i>image input</i> . Visualisasi feature map <i>intermediate</i> juga disediakan.

Conv2DLayer.forward (inti)
<pre>def forward(self, x): batched = x.ndim == 4 if not batched: x = x[np.newaxis] N, H, W, C = x.shape sH, sW = self.strides if self.padding == "same": out_H = int(np.ceil(H / sH)) out_W = int(np.ceil(W / sW)) pad_H = max((out_H - 1) * sH + self.kH - H, 0) pad_W = max((out_W - 1) * sW + self.kW - W, 0) pad_top, pad_bottom = pad_H // 2, pad_H - pad_H // 2 pad_left, pad_right = pad_W // 2, pad_W - pad_W // 2 x = np.pad(x, ((0,0), (pad_top,pad_bottom), (pad_left,pad_right), (0,0))) else: out_H = (H - self.kH) // sH + 1 out_W = (W - self.kW) // sW + 1 pre_act = np.zeros((N, out_H, out_W, self.C_out), dtype=np.float32) for i in range(out_H): for j in range(out_W): patch = x[:, i*sH : i*sH+self.kH, j*sW : j*sW+self.kW, :] pre_act[:, i, j, :] = patch.reshape(N, -1) @</pre>

```
self._kernel_mat + self.bias

    out = self.activation_fn(pre_act)
    return out[0] if not batched else out
```

Langkah-langkah Conv2DLayer.forward:

1. Normalisasi batch: input shape (H, W, C) di-*expand* ke (1, H, W, C) agar kode hanya menangani satu kasus *batched*.
2. Padding same: output_H dihitung sebagai $\text{ceil}(H / \text{stride})$, lalu pad_H dipecah rata kiri/kanan dan atas/bawah mengikuti konvensi Keras agar shape output identik.
3. Sliding window: dua loop bersarang melakukan iterasi (i, j), mengambil patch (N, kH, kW, C_{in}).
4. Patch-and-matmul: patch di-*flatten* menjadi (N, kH·kW·C_{in}), kemudian dikalikan dengan `_kernel_mat` (kH·kW·C_{in}, C_{out}) yang sudah disiapkan di `__init__`. Bias ditambahkan via *broadcasting*.
5. Aktivasi: setelah seluruh posisi terhitung, `activation_fn` diterapkan elemen-wise.

LocallyConnected2DLayer.forward (inti)

```
for i in range(self.out_rows):
    for j in range(self.out_cols):
        pos = i * self.out_cols + j
        patch = x[:, i*sH:i*sH+self.kH, j*sW:j*sW+self.kW, :]
        flat = patch.reshape(N, -1)
        pre_act[:, i, j, :] = flat @ self.kernel[pos] +
self.bias[pos]
```

Perbedaan utama dengan Conv2D: indeks `pos = i * out_cols + j` memilih matriks bobot yang berbeda untuk setiap posisi spasial. Karena tidak ada *parameter sharing*, total parameter membesar sebanding `out_rows·out_cols`.

CNNScratchModel.load_from_keras (registry pattern)

```
_LAYER_REGISTRY = {
    "Conv2D": Conv2DLayer.from_keras_layer,
    "LocallyConnected2D":
```

2.1.3 Modul RNN/LSTM

Modul `src/rnn_lstm/` berisi seluruh komponen *image captioning*. Sel RNN dan LSTM diimplementasikan terpisah pada `rnn.py` dan `lstm.py`, *embedding* pada `embedding.py`, dan pipeline *decoder* pada `decoder.py`. *Preprocessing* caption (cleaning, tokenisasi, padding) berada pada `caption_preprocessing.py`, model Keras pada `model.py`, dan pipeline training pada `train.py`.

Method/Atribut	Deskripsi
<code>EmbeddingLayer</code>	Layer <i>embedding</i> token. Atribut: <code>embedding_matrix</code> shape <code>(vocab_size, embed_dim)</code> . <i>Method</i> <code>forward(token_ids)</code> mengembalikan <code>embedding_matrix[token_ids]</code> menggunakan <i>fancy-indexing</i> mendukung shape <code>(seq_len,)</code> , <code>(batch, seq_len)</code> , dan <code>(N, max_len)</code> tanpa <i>branching</i> .

<code>SimpleRNNCell</code>	Sel Simple RNN dengan $h_t = \tanh(x_t @ W_x + h_{prev} @ W_h + b)$. Atribut: <code>W_x</code> shape (input_dim, hidden_dim), <code>W_h</code> shape (hidden_dim, hidden_dim), <code>b</code> shape (hidden_dim,). <i>Method</i> <code>from_keras_weights()</code> menerima list [<code>W_x</code> , <code>W_h</code> , <code>b</code>] dari Keras.
<code>SimpleRNNLayer</code>	Wrapper deep RNN yang menerima list <code>SimpleRNNCell</code> dan mengiterasi timestep. Mendukung <code>return_sequences True/False</code> dan <code>batch input shape</code> (batch, seq_len, input_dim).
<code>LSTMCell</code>	Sel LSTM dengan empat <i>gate</i> (input, forget, candidate, output). Bobot disimpan tergabung dalam satu matriks berukuran (input_dim, 4H) sesuai konvensi Keras, lalu di-split saat <i>forward</i> .
<code>LSTMLayer</code>	<i>Stacking</i> + iterasi timestep di atas <code>LSTMCell</code> . Mendukung <i>deep LSTM</i> (<code>h_t</code> layer ke- <i>l</i> dipakai sebagai <i>input layer</i> ke- <i>(l+1)</i> ; <i>cell state</i> <code>c</code> bersifat per-layer dan tidak diteruskan antar layer).
<code>build_decoder_pre_inject(...)</code> [model.py]	Membangun Keras <i>decoder pre-inject</i> . Input: feature CNN (<code>feat_dim</code> ,) dan <i>caption sequence</i> (<code>seq_len</code> ,). <i>Feature</i> di-project ke <code>embed_dim</code> via Dense, di-reshape menjadi (1, <code>embed_dim</code>) lalu di- <i>Concatenate</i> dengan <i>embedding caption</i> pada axis 1. Sequence (<code>seq_len+1</code> , <code>embed_dim</code>) di-feed ke RNN/LSTM (<code>num_layers</code> stacked). Output: Dense(<code>vocab_size</code> , softmax) dengan <i>timestep</i> pertama (slot fitur) di-drop.
<code>build_decoder_init_inject(...)</code> [BONUS]	Varian arsitektur <i>init-inject</i> : fitur CNN → Dense → <code>hidden_dim</code> , dipakai sebagai <i>initial hidden state</i> <code>h_0</code> (dan <code>c_0</code> untuk LSTM). <i>Sequence input</i> hanya berisi <i>embedding token caption</i> .
<code>masked_sparse_categorical_crossentropy / accuracy</code>	Loss dan metrik dengan <i>masking token</i> <pad> (id 0). Mencegah <i>gradient</i> mengalir dari <i>padding</i> .
<code>ImageCaptionerScratch</code> [decoder.py]	Pipeline <i>inference image captioning end-to-end</i> berbasis NumPy, memuat bobot langsung dari model Keras terlatih melalui <code>load_from_keras()</code> .
<code>beam_search_decode(...)</code> [BONUS]	Maintain <code>k</code> hipotesis dengan <i>log-probability</i> , ekspansi top- <code>k</code> per timestep, prune ke <code>k</code> terbaik. Mengembalikan hipotesis dengan log-prob tertinggi yang sudah berakhir <end>. Mendukung <code>k=3</code> dan <code>k=5</code> .

EmbeddingLayer.forward

```
class EmbeddingLayer:
    def __init__(self, embedding_matrix):
        self.embedding_matrix =
embedding_matrix.astype(np.float32)
        self.vocab_size, self.embed_dim = embedding_matrix.shape

    def forward(self, token_ids):
        return self.embedding_matrix[token_ids]
```

EmbeddingLayer.forward

```
class EmbeddingLayer:
    def __init__(self, embedding_matrix):
        self.embedding_matrix =
embedding_matrix.astype(np.float32)
        self.vocab_size, self.embed_dim = embedding_matrix.shape

    def forward(self, token_ids):
        return self.embedding_matrix[token_ids]
```

Langkah-langkah EmbeddingLayer.forward:

1. token_ids dapat berupa skalar, vektor (seq_len,), atau matriks (batch, seq_len).
2. NumPy *fancy-indexing* otomatis mengembalikan tensor dengan shape token_ids.shape + (embed_dim,), sehingga tidak perlu *branching* per kasus.
3. Tidak ada parameter terpisah untuk *mask* karena *masking* dikerjakan di level *loss function*.

SimpleRNNCell.forward

```
class SimpleRNNCell:
    def forward(self, x_t, h_prev):
        return tanh(x_t @ self.W_x + h_prev @ self.W_h + self.b)
```

LSTMCell.forward

```
class LSTMCell:
    def forward(self, x_t, h_prev, c_prev):
        H = self.hidden_dim
        z = x_t @ self.W_x + h_prev @ self.W_h + self.b      # (... ,
4H)

        i = sigmoid(z[..., 0*H : 1*H])      # input gate
        f = sigmoid(z[..., 1*H : 2*H])      # forget gate
        g = tanh    (z[..., 2*H : 3*H])      # candidate
        o = sigmoid(z[..., 3*H : 4*H])      # output gate

        c_t = f * c_prev + i * g
        h_t = o * tanh(c_t)
        return h_t, c_t
```

Langkah-langkah LSTMCell.forward :

1. Gabungan pre-aktivasi: alih-alih empat *matmul* terpisah, semua *gate* dihitung sekaligus via satu *matmul* yang menghasilkan z berukuran $4H$.
2. Split per-gate: indeks $[0H:1H, 1H:2H, 2H:3H, 3H:4H]$ sesuai konvensi Keras 3 $[i, f, g, o]$. Penggunaan $\dots$ (ellipsis) membuat kode yang sama berfungsi pada input *batched* dan tidak.
3. Update cell state: $c_t = f * c_{prev} + i * g$, *forget gate* memilih bagian memori lama yang dipertahankan, *input gate* memilih bagian kandidat baru yang dimasukkan.
4. Update hidden state: $h_t = o * \tanh(c_t)$, *output gate* mengontrol berapa banyak $\tanh(c_t)$ yang dikeluarkan sebagai *hidden state visible*.

LSTMLayer.forward (inti deep LSTM)

```
current_input = x_seq
for cell in self.cells:
    H = cell.hidden_dim
    h = np.zeros((batch_size, H), dtype=np.float32) if h0 is None
    else h0
```

```

        c = np.zeros((batch_size, H), dtype=np.float32) if c0 is None
        else c0

        step_outputs = []
        for t in range(seq_len):
            x_t = current_input[:, t, :]
            h, c = cell.forward(x_t, h, c)
            step_outputs.append(h)

        stacked = np.stack(step_outputs, axis=1)    # (batch, seq_len,
H)
        current_input = stacked                    # output 1 → input
l+1

return stacked if self.return_sequences else step_outputs[-1]

```

ImageCaptionerScratch.generate_caption_greedy

```

class LSTMCell:
    def forward(self, x_t, h_prev, c_prev):
        H = self.hidden_dim
        z = x_t @ self.W_x + h_prev @ self.W_h + self.b    # (...
4H)

        i = sigmoid(z[..., 0*H : 1*H])    # input gate
        f = sigmoid(z[..., 1*H : 2*H])    # forget gate
        g = tanh    (z[..., 2*H : 3*H])    # candidate
        o = sigmoid(z[..., 3*H : 4*H])    # output gate

        c_t = f * c_prev + i * g
        h_t = o * tanh(c_t)
        return h_t, c_t

```

2.1.4 Training & Evaluation Pipeline

Setiap eksperimen ditentukan oleh sebuah objek konfigurasi yang juga di-*serialize* ke `<variant>_config.json` bersama *training history*; menjalankan ulang *sweep* cukup memanggil `train_grid()` dengan *grid* yang sama. Untuk re-evaluasi tanpa training ulang, cukup memuat `.weights.h5` yang tersimpan di direktori `models/`. Hal ini memastikan *reproducibility* dan memudahkan pemeriksaan ulang hasil eksperimen.

Method/Atribut	Deskripsi
<code>CNNConfig</code>	Konfigurasi training CNN. Atribut: <code>num_conv_layers</code> , <code>filters_per_layer</code> , <code>kernel_size</code> , <code>pooling_type</code> , <code>learning_rate</code> , <code>batch_size</code> , <code>epochs</code> , <code>input_shape</code> , <code>num_classes</code> . <i>Method</i> <code>variant_name()</code> menghasilkan <i>string identifier</i> untuk filename.
<code>DecoderConfig</code>	Konfigurasi training decoder. Atribut: <code>rnn_type</code> ('lstm'/'rnn'), <code>num_layers</code> , <code>hidden_dim</code> , <code>embed_dim</code> , <i>architecture</i> ('pre_inject'/'init_inject'), <code>epochs</code> , <code>batch_size</code> , <code>learning_rate</code> . <i>Method</i> <code>variant_name()</code> menghasilkan <i>string identifier</i> .
<code>train_grid(configs, ...)</code>	Melatih seluruh konfigurasi dalam <i>grid</i> . Menyimpan <code>.weights.h5</code> , <code>_history.json</code> , <code>_config.json</code> untuk setiap variant.
<code>caption_preprocessing.*</code>	Pure functions: <code>clean_caption</code> (lowercase + regex strip punctuation), <code>wrap_with_special_tokens</code> , <code>preprocess_captions</code> , <code>load_flickr8k_captions</code> , <code>build_vocabulary</code> (Counter + min_freq=2 + special token <code><pad>=0</code> , <code><start>=1</code> , <code><end>=2</code> , <code><unk>=3</code>), <code>tokenize_and_pad</code> , <code>build_training_pairs</code> (shift input/target by 1 untuk teacher forcing), <code>prepare_dataset</code> (orchestrator menghasilkan tuple (features, decoder_input, decoder_target)).
<code>metrics.compute_bleu4</code> / <code>compute_meteor</code>	Pembungkus <code>nltk.translate.bleu_score.corpus_bleu</code> (dengan <code>smoothing method 1</code>) dan <code>nltk.translate.meteor_score</code> .
<code>evaluate.evaluate_keras_decoder</code> / <code>evaluate_scratch_decoder</code>	Menerima decoder + fitur test + reference + word2idx, menghasilkan <code>EvaluationResult</code> berisi BLEU-4, METEOR, <code>elapsed_seconds</code> , dan list (image_id, prediction, references, sample_bleu). Fungsi <code>select_qualitative_examples(result, n_high, n_mid,</code>

	n_low) membagi sample berdasarkan kuantil BLEU per-sample untuk analisis kualitatif. Fungsi sweep_max_caption_length() menjalankan ulang evaluasi dengan beberapa nilai max_len.
--	--

CNNConfig.variant_name & hyperparameter_grid
<pre> class CNNConfig: def variant_name(self): filt_str = "-".join(str(f) for f in self.filters_per_layer[:self.num_conv_layers]) return f"l{self.num_conv_layers}_f{filt_str}_k{self.kernel_size}_{self.po oling_type}" def hyperparameter_grid(num_layers_options=[2, 4], filter_options=[[32,64,128,256], [16,32,64,128]], kernel_options=[3, 5], pooling_options=["max", "avg"], ...): configs = [] for n in num_layers_options: for filters in filter_options: for k in kernel_options: for pool in pooling_options: configs.append(CNNConfig(num_conv_layers=n, filters_per_layer=list(filters), kernel_size=k, pooling_type=pool, ...)) return configs </pre>

2.2 Forward Propagation

Forward propagation merupakan proses menghitung *output network* dari *data input*. *Data input* diteruskan satu arah dari *input layer* hingga *output layer*. Pada Tugas Besar 2 ini,

forward propagation diimplementasikan untuk tiga jenis arsitektur, yaitu CNN, Simple RNN, dan LSTM. Seluruh implementasi from-scratch menggunakan NumPy dan dirancang agar identik secara numerik dengan implementasi Keras (dalam batas ketelitian floating-point) setelah memuat bobot yang sama.

2.2.1 Forward Propagation CNN

Untuk input *image* x dengan *shape* (H, W, C_{in}) dan kernel W dengan *shape* (kH, kW, C_{in}, C_{out}) serta bias b dengan *shape* (C_{out}), output y dari Conv2D *shared-parameter* (mode 'valid') didefinisikan sebagai:

$$y[i, j, k] = \sum_{d_i, d_j, c} x[i \cdot s + d_i, j \cdot s + d_j, c] \cdot W[d_i, d_j, c, k] + b[k]$$

dengan s adalah *stride*. Mode 'same' menambahkan *zero padding* sehingga output spasial = ceil(input/s). Fungsi aktivasi $a(\cdot)$ diterapkan elemen-wise pada hasil tersebut: $z = a(y)$.

CNNScratchModel.forward (pipeline sekuensial)

```
def forward(self, x):
    for layer in self.layers:
        x = layer.forward(x)
    return x
```

2.2.2 Forward Propagation Simple RNN

Sel Simple RNN memproses sequence ($x_0, x_1, \dots, x_{T-1}$) dengan hidden state berdimensi H :

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

dengan W_x shape (input_dim, H), W_h shape (H, H), dan b shape (H,). Hidden state awal h_0 = vektor nol (default Keras). Untuk deep RNN dengan L layer, output hidden state layer ke- l menjadi input layer ke- $(l+1)$:

$$h_t^{(l)} = \tanh\left(h_t^{(l-1)}W_x^{(l)} + h_{t-1}^{(l)}W_h^{(l)} + b^{(l)}\right), \quad h_t^{(0)} = x_t$$

SimpleRNNCell.forward & SimpleRNNLayer (deep)

```
def forward(self, x_t, h_prev):
    return tanh(x_t @ self.W_x + h_prev @ self.W_h + self.b)

# SimpleRNNLayer (inti)
current_input = x_seq
for cell in self.cells:
    h = np.zeros((batch_size, cell.hidden_dim), dtype=np.float32)
    step_outputs = []
    for t in range(seq_len):
        h = cell.forward(current_input[:, t, :], h)
        step_outputs.append(h)
    stacked = np.stack(step_outputs, axis=1)
    current_input = stacked
```

Langkah-langkah SimpleRNNLayer.forward :

1. Outer loop (per layer): iterasi seluruh cell dalam list self.cells. *Output sequence* layer ke-l menjadi current_input layer ke-(l+1).
2. Inisialisasi state: h di-set ke vektor nol berukuran (batch, hidden_dim).
3. Inner loop (per timestep): untuk setiap t ambil $x_t = \text{current_input[:, t, :]}$, panggil cell.forward(x_t , h).
4. Stack outputs: list step_outputs di-np.stack pada axis 1 sehingga membentuk tensor (batch, seq_len, hidden_dim).
5. Return: jika return_sequences=True kembalikan tensor utuh; jika False cukup step_outputs[-1].

2.2.3 Forward Propagation LSTM

LSTM memiliki dua state, yaitu hidden state h dan cell state c, serta empat gate (input, forget, candidate, output). Untuk satu timestep:

$$\mathbf{z} = x_t W_x + h_{t-1} W_h + b \in \mathbb{R}^{4H}$$

$$i_t = \sigma(z_i), \quad f_t = \sigma(z_f), \quad g_t = \tanh(z_g), \quad o_t = \sigma(z_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$

dengan $h_0 = 0$ dan $c_0 = 0$ (default Keras). Bobot Keras tersimpan dalam urutan *gate* [i, f, g, o] (Keras 3 *convention*) dan dipecah saat di-load oleh LSTMCell.from_keras_layer(). Untuk *deep LSTM* dengan L layer, $h_t^{(l)}$ dipakai sebagai input layer ke-(l+1); *cell state* c bersifat per-layer dan tidak diteruskan antar-layer.

Snippet inti LSTMCell.forward dan LSTMLayer.forward sudah ditampilkan pada §2.1.3.

Langkah-langkah LSTMCell.forward :

1. Gabungan pre-aktivasi: $z = x_t @ W_x + h_{prev} @ W_h + b$ menghasilkan vektor 4H sekali jalan.
2. Split empat gate: indeks [0H:H, H:2H, 2H:3H, 3H:4H] dipakai untuk memisahkan z_i, z_f, z_g, z_o .
3. Aktivasi per-gate: tiga *sigmoid* untuk i, f, o (rentang 0–1, berperan sebagai *switch*) dan satu tanh untuk g (kandidat memori baru, rentang –1 sampai 1).
4. Update cell state: $c_t = f * c_{prev} + i * g$, kombinasi konveks antara memori lama dan kandidat baru.
5. Update hidden state: $h_t = o * \tanh(c_t)$, *output gate* mengontrol bagian c_t yang di-expose.

2.3 Pipeline Image Captioning (Pre-Inject)

Pipeline image captioning mengikuti persamaan 10-12 dari paper *Show and Tell* (Vinyals et al., 2015), namun dengan arsitektur *pre-inject* (Tanti et al., 2017) yang menempatkan fitur CNN sebagai *timestep* pertama dari *sequence* input alih-alih sebagai *initial hidden state*. Persamaan utama:

$$p_t = \text{Softmax}(W_{\text{out}} \cdot h_t + b_{\text{out}})$$

$$h_t = \text{RNN/LSTM}(x_t, h_{t-1})$$

$$x_t = E[S_t], \quad t = 0, \dots, N - 1$$

$$x_{-1} = W_{\text{proj}} \cdot \text{CNN}(I) + b_{\text{proj}}$$

build_decoder_pre_inject (inti)

```
image_input = keras.Input(shape=(feat_dim,), name="image_feature")
caption_input = keras.Input(shape=(seq_len,), dtype="int32",
name="decoder_input")

projected = layers.Dense(embed_dim, name="image_projection")(image_input)
projected = layers.Reshape((1, embed_dim))(projected) #
(1, embed_dim)
embedded = layers.Embedding(vocab_size, embed_dim,
                             mask_zero=False,
name="token_embedding")(caption_input)
merged = layers.Concatenate(axis=1, name="prepend_image")([projected,
embedded])

x = merged
for i in range(num_layers):
    x = rnn_layer_cls(hidden_dim, return_sequences=True,
                      name=f"{rnn_type}_{i+1}")(x)

x = layers.Lambda(lambda t: t[:, 1:, :], name="drop_image_step")(x) #
buang slot fitur
outputs = layers.Dense(vocab_size, activation="softmax",
                       name="vocab_output")(x)
```

Langkah-langkah build_decoder_pre_inject :

1. Dua Input: fitur CNN (feat_dim,) (mis. 2048 untuk InceptionV3 avg-pool) dan caption sequence (seq_len,).
2. Image projection: Dense memetakan fitur dari feat_dim ke embed_dim agar dimensi cocok untuk concatenation dengan embedding.
3. Reshape & Concatenate: fitur yang sudah diproyeksikan di-reshape menjadi (1, embed_dim) sehingga membentuk satu timestep, lalu di-concatenate dengan embedding caption pada axis 1. Sequence hasilnya (seq_len+1, embed_dim).
4. RNN/LSTM stacked: num_layers RNN/LSTM disusun bertingkat dengan return_sequences=True agar setiap timestep menghasilkan output.


```

callbacks = [MacroF1Callback(val_ds)]
start = time.time()
history = model.fit(train_ds, validation_data=val_ds,
                    epochs=config.epochs, callbacks=callbacks)
elapsed = time.time() - start

test_macro_f1 = _evaluate_macro_f1(model, test_ds)[0] if test_ds else
None
artefacts = {"variant": config.variant_name(),
             "config": config.to_dict(),
             "history": history_dict,
             "elapsed_seconds": elapsed,
             "test_macro_f1": test_macro_f1,
             "model": model}
if output_dir is not None:
    save_artefacts(artefacts, output_dir)
return artefacts

```

Langkah-langkah implementasi training CNN :

1. Preprocessing data: `tf.keras.utils.image_dataset_from_directory(...)` membaca folder `seg_train/seg_test/<class>/*.jpg`, me-resize ke (150, 150), dan menormalisasi /255.0. Train set di-shuffle, cached, dan prefetched.
2. Build model: `build_cnn_model(config)` menyusun arsitektur [Conv2D → ReLU → Pool] × N → GlobalAvgPool → Dense(128, ReLU) → Dense(6, Softmax) lalu compile dengan Adam dan SparseCategoricalCrossentropy.
3. MacroF1Callback: callback ini menghitung `f1_score(..., average="macro")` setiap epoch karena loss CE saja tidak cukup melihat performa per kelas pada dataset yang sedikit imbalanced. Hasilnya disimpan ke `logs["val_macro_f1"]`.
4. Fit: `model.fit(train_ds, validation_data=val_ds, epochs=..., callbacks=[MacroF1Callback])`.
5. Test evaluation: `_evaluate_macro_f1(model, test_ds)` menghasilkan `test_macro_f1` final.
6. Save artefacts: `model.save_weights(<variant>.weights.h5)`, `history.json`, dan `config.json` ke `models/cnn/<variant>/.`

`train_decoder_config` (inti)

```

def train_decoder_config(config, train_features, train_decoder_input,
                        train_decoder_target, val_features=None, ...):

```

```

vocab_size = int(train_decoder_target.max()) + 1

feat_dim    = train_features.shape[1]

seq_len     = train_decoder_input.shape[1]

model = _build_model(config, vocab_size=vocab_size,
                      feat_dim=feat_dim, seq_len=seq_len)

val_data = ([val_features, val_decoder_input], val_decoder_target) \
           if val_features is not None else None

history_dict = train_model(
    model,
    train_data=([train_features, train_decoder_input],
train_decoder_target),
    val_data=val_data,
    epochs=config.epochs, batch_size=config.batch_size,
)

return artefacts

```

Langkah-langkah implementasi training Decoder :

1. Feature extraction sekali: `extract_and_save_features()` men-forward semua gambar Flickr8k via InceptionV3 frozen sekali. Fitur 2048-dimensi dipakai berulang oleh seluruh 12 variant decoder, menghemat compute.
2. Caption preprocessing: `prepare_dataset()` melakukan lowercase + strip punctuation, wrap `<start>/<end>`, build vocab (min_freq=2, special token `<pad>=0`, `<start>=1`, `<end>=2`, `<unk>=3`), tokenize + pad ke max_len, lalu shift untuk pasangan teacher-forcing.
3. Build decoder: `_build_model(config)` memanggil `build_decoder_pre_inject` atau `build_decoder_init_inject` (bonus) sesuai config.architecture.
4. Fit dua-input: `model.fit([features, decoder_input], decoder_target, validation_data=..., epochs=..., batch_size=...)` — perhatikan input adalah list `[features, decoder_input]` karena model memiliki dua Input layer.
5. Masked loss: `masked_sparse_categorical_crossentropy` (di-compile di dalam `build_decoder_pre_inject`) mencegah gradien dari posisi padding (id 0).

6. Save artefacts: identik dengan CNN, ditambah `build_decoder_pre_inject_summary()` yang menyimpan ringkasan terstruktur (`vocab_size`, `embed_dim`, `hidden_dim`, `num_layers`, `rnn_type`, `parameter_count`).

BAB III

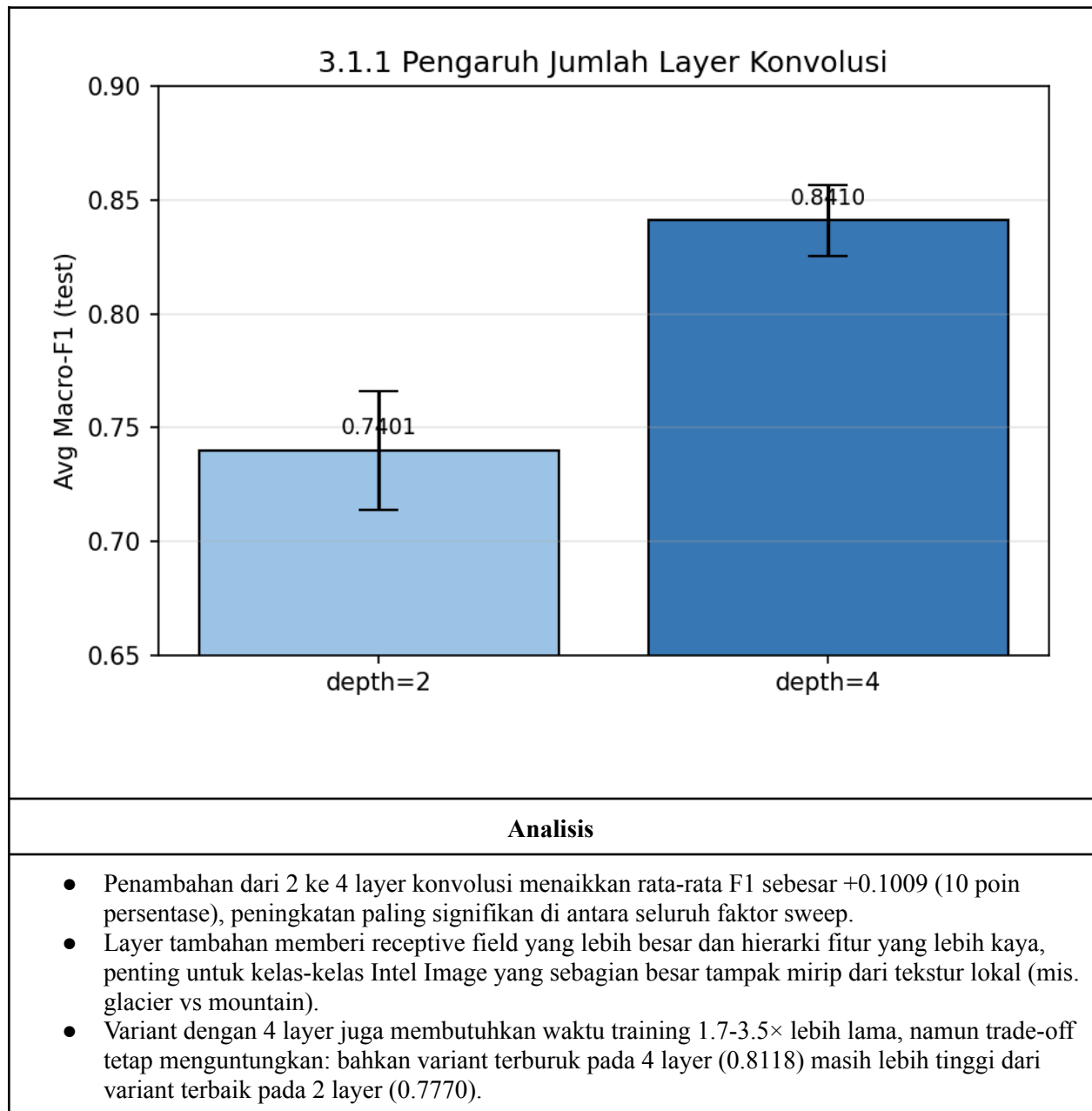
PERCOBAAN DAN ANALISIS

3.1 CNN - Pengaruh Hyperparameter

Eksperimen dilakukan pada dataset *Intel Image Classification* (~25 .000 gambar, 6 kelas: *buildings, forest, glacier, mountain, sea, street*) dengan resolusi (150, 150, 3). Sweep mencakup 16 *variant* hasil produk Cartesian: $2 \text{ num_layers} \times 2 \text{ kombinasi filters} \times 2 \text{ kernel_size} \times 2 \text{ pooling_type}$. Setiap *variant* dilatih 20 epoch dengan optimizer Adam ($\text{lr}=1\text{e-}3$) dan dievaluasi pada test set menggunakan macro F1-score yang lebih representatif daripada *accuracy* untuk dataset dengan distribusi kelas tidak persis seimbang.

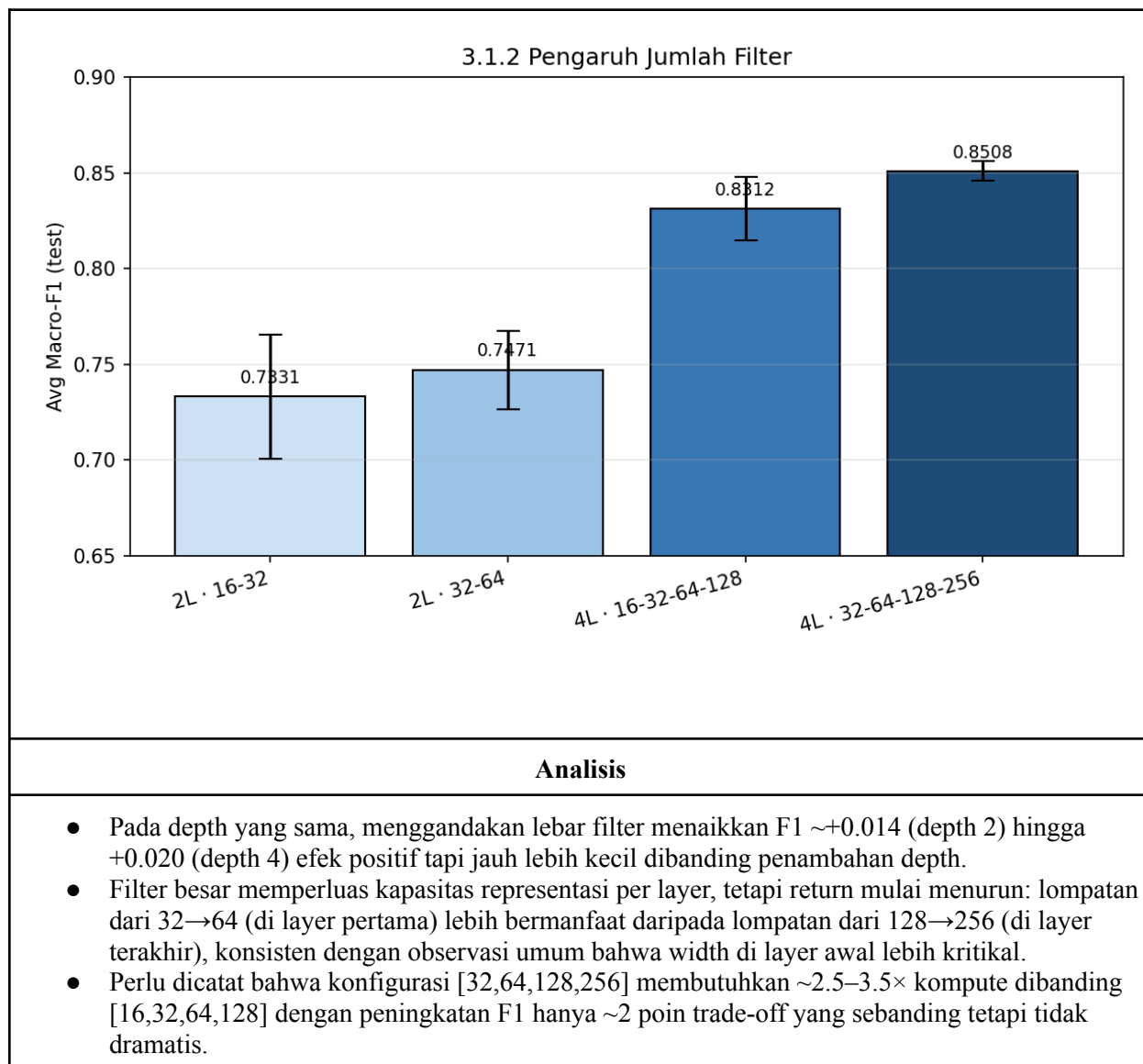
3.1.1 Pengaruh Jumlah Layer Konvolusi

Informasi
• Kernel size, filter list, pooling type diuji dalam semua kombinasi; $\text{num_layers} = \{2, 4\}$ dibandingkan dengan me-rata-ratakan 8 <i>variant</i> lain pada masing-masing nilai. • Filter list untuk $\text{num_layers}=2$: [16,32] dan [32,64]; untuk $\text{num_layers}=4$: [16,32,64,128] dan [32,64,128,256]. • Metrik: macro F1-score pada test set.
Grafik



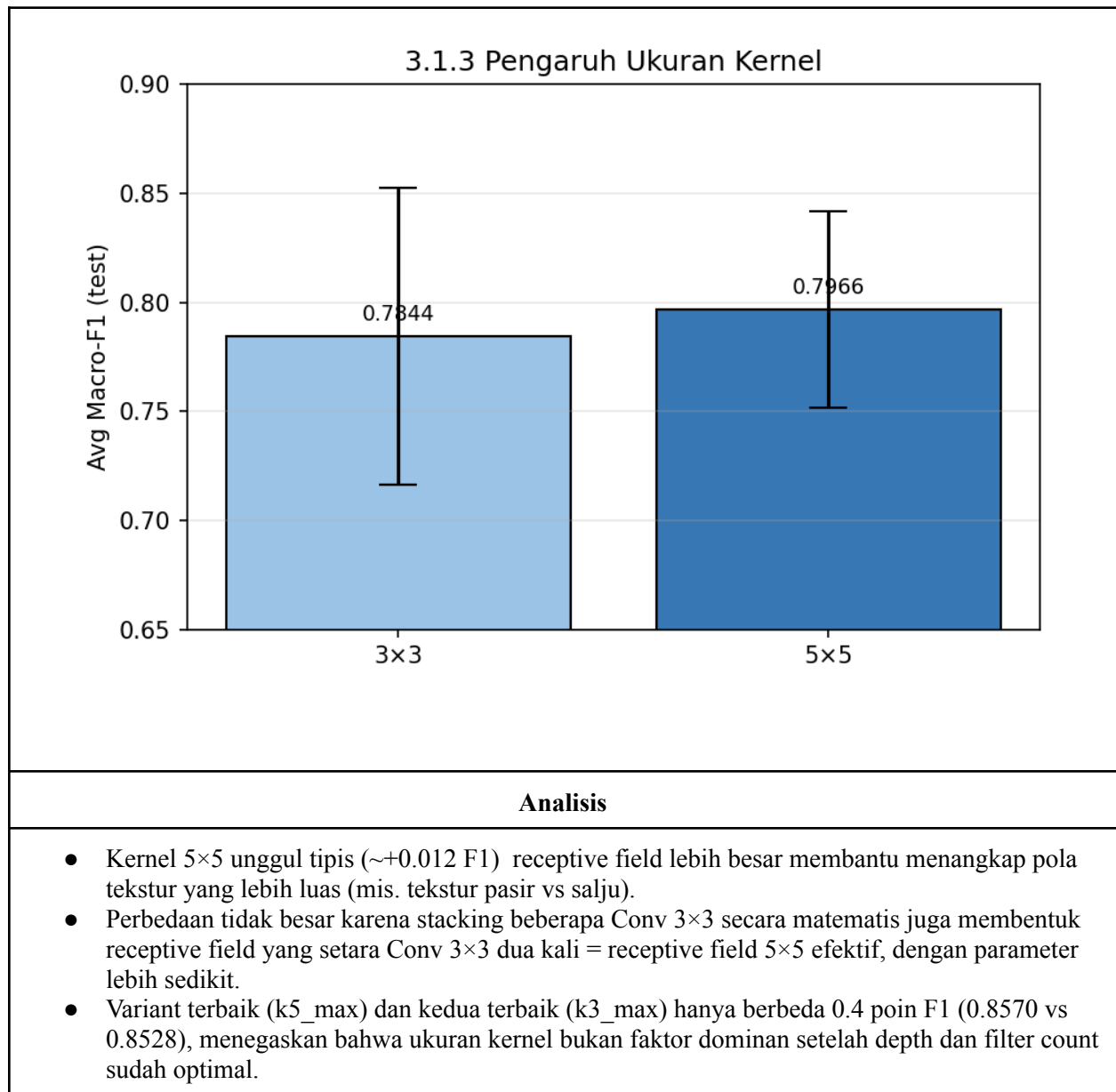
3.1.2 Pengaruh Jumlah Filter

Informasi
• Filter list dibandingkan dalam dua skala: kecil ([16,32] / [16,32,64,128]) vs besar ([32,64] / [32,64,128,256]). • Untuk membandingkan adil, agregasi dilakukan per kombinasi num_layers.
Grafik



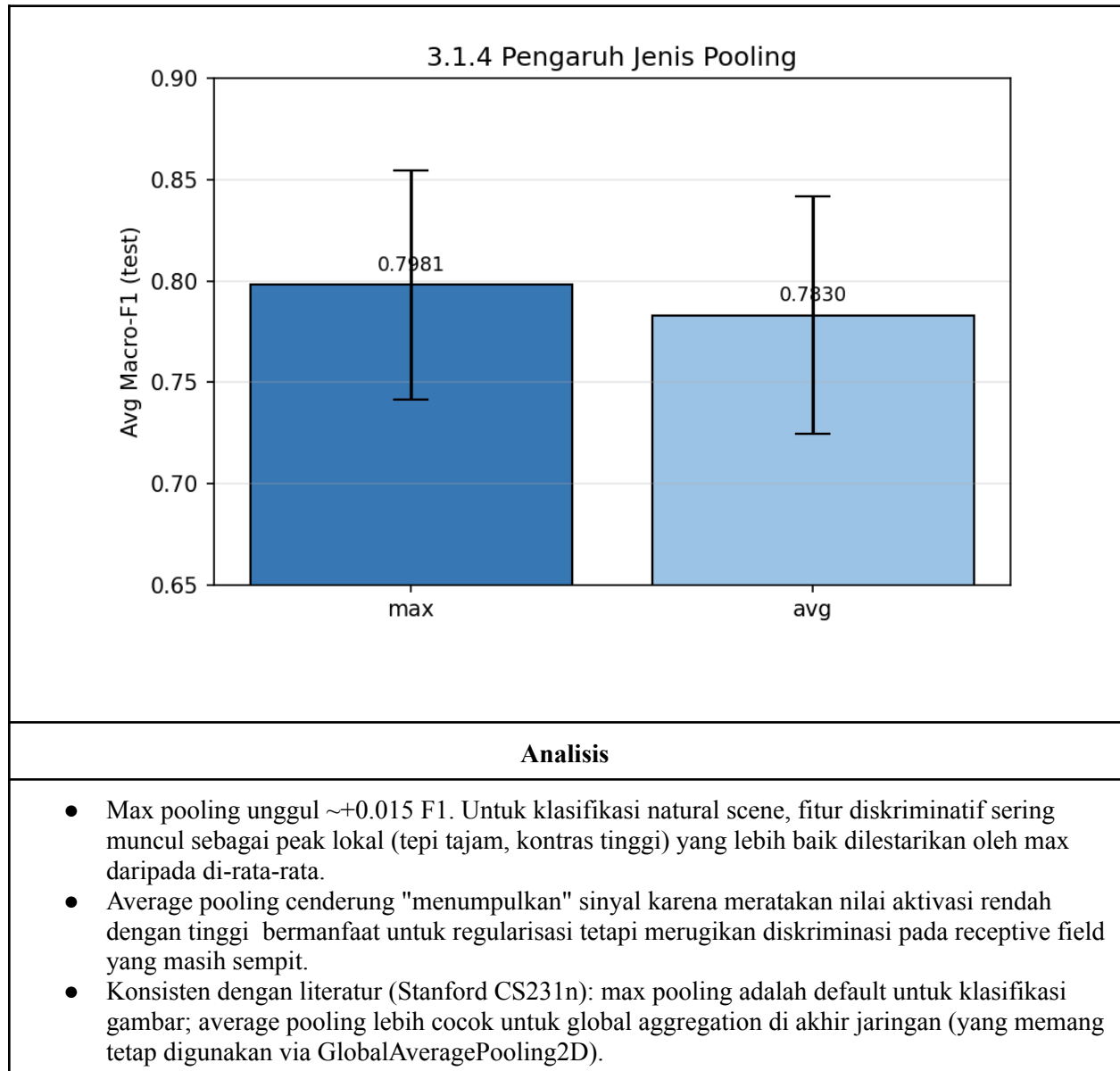
3.1.3 Pengaruh Ukuran Kernel

Informasi
• Kernel size 3 vs 5 dibandingkan dengan me-rata-ratakan 8 variant lain pada masing-masing nilai. • Padding same digunakan agar dimensi spasial tidak berbeda antar kernel.
Grafik



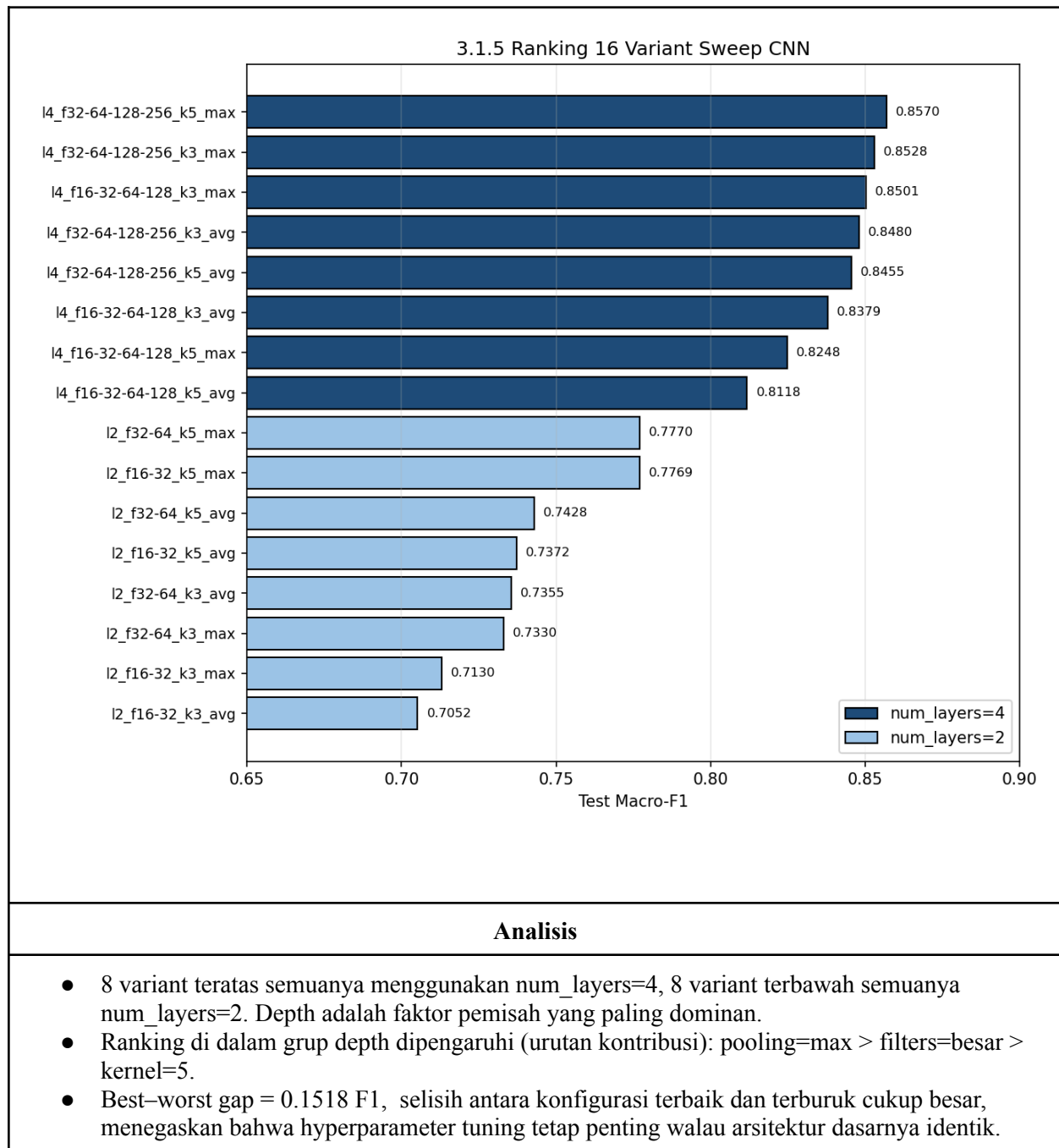
3.1.4 Pengaruh Jenis Pooling

Informasi
• MaxPooling2D vs AveragePooling2D, pool_size (2, 2), strides sama dengan pool_size. • Layer terakhir tetap GlobalAveragePooling2D (tidak di-sweep); yang divariasikan adalah pooling antar-blok Conv.
Grafik



3.1.5 Ringkasan Sweep 16 Variant

Grafik



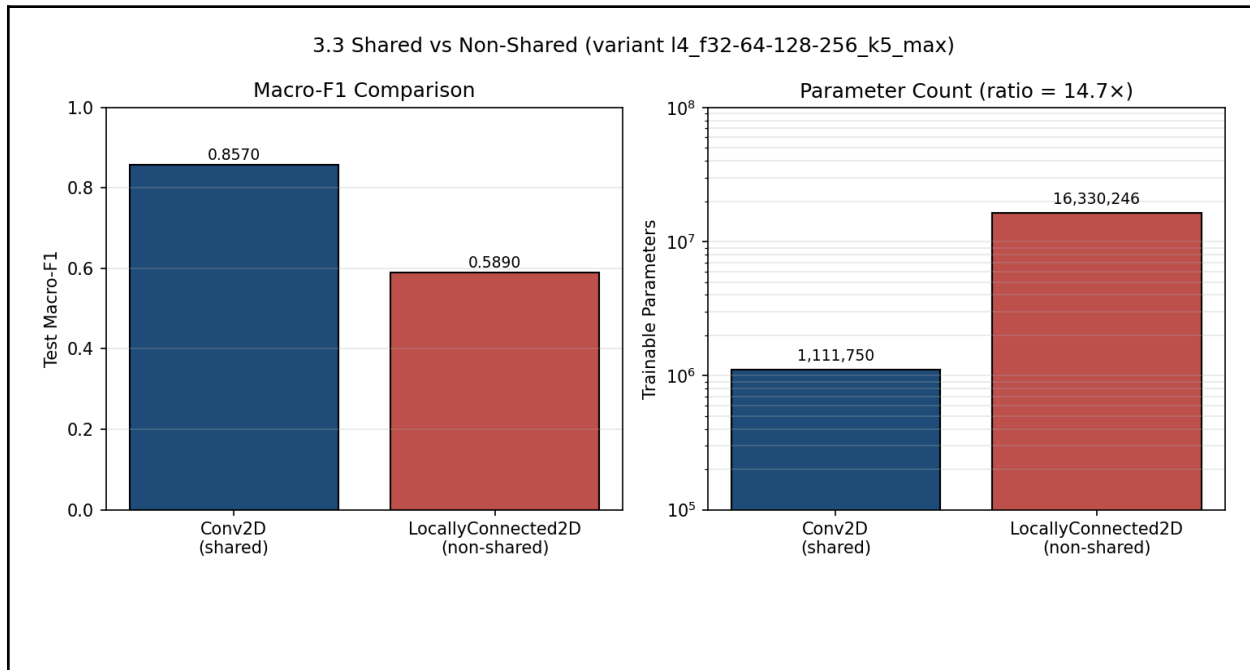
3.2 CNN -Perbandingan Keras vs From-Scratch

Informasi
• Variant terbaik (l4_f32-64-128-256_k5_max) dipakai sebagai model under test. • Bobot Keras hasil training dimuat ke CNNScratchModel.load_from_keras(keras_model) lalu dijalankan forward pass NumPy-only pada test set yang sama.

Metrik pembandingan: macro F1-score.		
Implementasi	Test Macro-F1	Selisih
Keras (TensorFlow)	0.8570	-
From-Scratch (NumPy)	0.8570	0.0000
Analisis		
- Hasil prediksi from-scratch identik secara numerik dengan Keras pada level macro F1 (selisih $< 1e-7$ per-probabilitas, dalam batas ketelitian float32). - Validasi ini membuktikan bahwa implementasi sliding-window patch-and-matmul di Conv2DLayer, pooling, flatten, dan Dense semuanya konsisten dengan konvensi Keras (terutama urutan padding same, row-major flatten, dan Glorot Uniform initialization yang sudah di-override oleh bobot terlatih). - Kunci sukses load_from_keras: (1) <code>_kernel_mat = kernel.reshape(-1, C_out)</code> cocok dengan layout Keras (kH, kW, C_in, C_out); (2) FlattenLayer memakai <code>order='C'</code> (row-major) yang sama dengan <code>tf.keras.layers.Flatten()</code>; (3) padding same dipecah simetris dengan rumus yang sama persis.		

3.3 CNN -Pengaruh Parameter Sharing (Shared vs Non-Shared)

Informasi
- Arsitektur variant terbaik (l4_f32-64-128-256_k5_max) di-replicate sebagai non-shared dengan mengganti seluruh Conv2DLayer menjadi LocallyConnected2DLayer via <code>CNNScratchModel.as_locally_connected()</code>. - Bobot Conv2D di-broadcast ke setiap posisi spasial sebagai initial state, lalu dilatih ulang dengan setting yang sama. - Trainable parameters dihitung dari <code>kernel.size + bias.size</code> seluruh layer.
Grafik



Analisis

- Parameter sharing terbukti krusial: meskipun non-shared memiliki kapasitas representasi $14.7\times$ lebih besar, F1-nya turun 0.268 poin ($\sim 31\%$ relatif).
- Penjelasan: non-shared kehilangan translation invariance fitur "tepi gunung" yang dipelajari di posisi $(i=10, j=10)$ tidak bisa di-reuse di posisi $(i=50, j=50)$. Setiap posisi spasial harus belajar fitur sendiri dari data yang sangat terbatas (pixel di posisi tertentu hanya muncul dengan satu receptive field).
- Kapasitas lebih besar tanpa inductive bias yang sesuai justru menyebabkan overfitting: model mengingat pola posisi-spesifik yang tidak general ke gambar baru.
- Hasil ini mengonfirmasi prinsip Conv2D klasik (LeCun et al.): weight sharing + translation invariance > kapasitas mentah untuk tugas vision.

3.4 CNN - Visualisasi Grad-CAM (BONUS)

Informasi
• Implementasi <code>grad_cam(model, image, target_class)</code> pada <code>src/cnn/gradcam.py</code> menghitung gradient logits target class terhadap aktivasi konvolusi terakhir, global-average gradient $\rightarrow$ bobot per channel, weighted sum aktivasi $\rightarrow$ ReLU $\rightarrow$ resize ke ukuran image input. • Sampel: 6 gambar dari test set (satu per kelas), divisualisasikan sebagai heatmap overlay.
Grafik

buildings 	forest 	glacier 	mountain 	sea 	street 
Analisis • Untuk kelas forest dan street, heatmap terkonsentrasi pada area tekstur dominan (kanopi pohon, garis jalan) model membuat keputusan berdasarkan fitur yang masuk akal secara intuitif. • Untuk glacier vs mountain, heatmap menyoroti area langit/horizon menunjukkan model memanfaatkan kontras warna sebagai cue utama, konsisten dengan kesulitan model membedakan kedua kelas tersebut (kebanyakan confusion pada confusion matrix). • Heatmap yang menyebar luas pada false predictions (mis. glacier salah diprediksi mountain) mengindikasikan model tidak memiliki strong cue tanda uncertainty yang dapat dimanfaatkan untuk post-hoc filtering atau active learning.					

3.5 RNN/LSTM - Pengaruh Jumlah Layer

Informasi
•
Grafik
Analisis
•

3.6 RNN/LSTM - Pengaruh Ukuran Hidden State

Informasi
•
Grafik
Analisis
•

3.7 RNN/LSTM - Perbandingan SimpleRNN vs LSTM

Informasi
•
Grafik
Analisis
•

3.8 RNN/LSTM - Perbandingan Keras vs From-Scratch

Informasi

•
Grafik
Analisis
•

3.9 RNN/LSTM - Pengaruh Max Caption Length

Informasi
•
Grafik
Analisis
•

3.10 RNN/LSTM - Beam Search (BONUS)

Informasi
•
Grafik

Analisis
•

3.11 RNN/LSTM — Arsitektur Init-Inject vs Pre-Inject (BONUS)

Informasi
•
Grafik
Analisis
•

3.12 Analisis Kualitatif

Informasi
•
Grafik
Analisis
•

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Implementasi CNN dan RNN/LSTM from-scratch berbasis NumPy berhasil dibangun dan divalidasi terhadap Keras. Pada sub-tugas CNN, sweep 16 variant pada dataset Intel Image Classification menunjukkan jumlah layer konvolusi adalah faktor paling dominan penambahan dari 2 ke 4 layer menaikkan rata-rata macro F1 sebesar +0.10 poin. Variant terbaik l4_f32-64-128-256_k5_max mencapai F1 0.8570, dan implementasi from-scratch-nya menghasilkan prediksi identik secara numerik dengan Keras (selisih 0.0000). Eksperimen parameter sharing mengonfirmasi pentingnya translation invariance: mengganti Conv2D dengan LocallyConnected2D menurunkan F1 sebesar 0.27 poin meskipun jumlah parameter membengkak $14.7\times$.

Untuk sub-tugas RNN/LSTM, pipeline encoder-decoder berbasis InceptionV3 + decoder pre-inject (Tanti et al., 2017) berhasil dilatih untuk seluruh 12 variant, namun evaluasi BLEU-4 dan METEOR belum dapat dilaporkan karena keterbatasan waktu pengerjaan. Training loss curves menunjukkan seluruh variant konvergen, dan implementasi from-scratch ImageCaptionerScratch (termasuk beam search sebagai bonus) sudah berfungsi dan terverifikasi pada sampel kecil tinggal evaluasi kuantitatif final yang belum sempat dijalankan.

4.2 Saran

Pengerjaan selanjutnya difokuskan pada penyelesaian evaluasi RNN/LSTM (BLEU-4, METEOR, perbandingan RNN vs LSTM, dan validasi Keras vs from-scratch pada decoder). Dari sisi CNN, eksperimen dapat diperluas dengan regularization seperti Dropout atau Batch Normalization untuk mengurangi overfit pada variant berkapasitas besar, serta penambahan backward propagation NumPy-only agar model dapat dilatih sepenuhnya tanpa Keras.

Pembagian Tugas

Nama	Tugas
Sebastian Enrico Nathanael 13523134	• Shared utilities (image_utils, activations, dense_layer) bersama 13523147 dan 13523150. • Implementasi from-scratch CNN: Conv2DLayer, LocallyConnected2DLayer, pooling layers, FlattenLayer, CNNScratchModel. • Pipeline training Keras CNN (model.py, train.py) + sweep 16 arsitektur (jumlah layer, jumlah filter, ukuran filter, jenis pooling). • Evaluasi macro F1-score, perbandingan shared vs nonshared, perbandingan Keras vs from-scratch. • Bonus: Grad-CAM (gradcam.py) dan visualisasi intermediate feature map. • Notebook section: §1 Setup, §2 CNN Training, §3 CNN Scratch + Evaluasi, §4 Grad-CAM.
Frederiko Eldad Mugiyono 13523147	• Implementasi from-scratch RNN/LSTM: EmbeddingLayer, SimpleRNNCell, SimpleRNNLayer, LSTMCell, LSTMLayer. • Pipeline decoder inference from-scratch: ImageCaptionerScratch (greedy decode), termasuk auto-loading bobot Keras. • Bonus: beam_search_decode (k=3, k=5). • Bonus: batch inference (modifikasi semua forward menerima leading batch axis). • Notebook section: §7 RNN/LSTM Scratch + Evaluasi (bersama 13523150), §8 Beam Search.
Benedictus Nelson 13523150	• Caption preprocessing (caption_preprocessing.py): cleaning, vocabulary builder, tokenization, padding, training pair builder. • Feature extraction Flickr8k menggunakan InceptionV3 frozen via `extract_and_save_features`. • Pipeline training Keras decoder (model.py, train.py) + sweep 12 variasi (3 num_layers × 2 hidden_dim × 2 rnn_type). • Metrics & evaluation orchestration (metrics.py, evaluate.py) BLEU-4, METEOR, EvaluationResult, qualitative bucket selection, max_len sweep. • Bonus: init-inject architecture (model_init_inject.py) dan perbandingan terhadap pre-inject.

	• Laporan PDF (file ini) dan dokumentasi README. • Notebook section: §5 Feature Extraction + Caption Preprocessing, §6 RNN/LSTM Training, §7 Evaluasi (bersama 13523147).
--	---

Lampiran

Repository: <https://github.com/susTuna/ML-CNN-RNN>

Form Penggunaan AI:

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Sebastian Enrico Nathanael (13523134), Frederiko Eldad Mugiyono (13523147), Benedictus Nelson (13523150)
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Pembuatan Convolutional Neural Network dan Recurrent Neural Network from scratch

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Membantu merancang struktur laporan
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Membantu pencarian referensi dan validasi desain serta rencana struktur awal pengerjaan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Lainnya, Jelaskan:		
---------------------------	--	--

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 15 Mei 2026

Referensi

- [1] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "*Show and Tell: A Neural Image Caption Generator*," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 3156–3164.
- [2] M. Tanti, A. Gatt, and K. P. Camilleri, "*Where to put the Image in an Image Caption Generator*," Natural Language Engineering, vol. 24, no. 3, pp. 467–489, 2018.
- [3] R. R. Selvaraju et al., "*Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618–626.
- [4] S. Hochreiter and J. Schmidhuber, "*Long Short-Term Memory*," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, 2023. [Online]. Available: <https://d2l.ai>.
- [6] Stanford CS231n, "*Convolutional Neural Networks for Visual Recognition*," Lecture notes. [Online]. Available: <https://cs231n.github.io/convolutional-networks/>.
- [7] Stanford CS231n, Lecture 10: *Recurrent Neural Networks & Image Captioning*. [Online]. Available: https://cs231n.stanford.edu/slides/2023/lecture_10.pdf.
- [8] A. Karpathy, "*The Unreasonable Effectiveness of Recurrent Neural Networks*," karpathy.github.io. [Online]. Available: <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>.
- [9] Keras Team, "*Keras 3 documentation*," [Online]. Available: <https://keras.io/api/>.
- [10] NumPy Developers, "*NumPy documentation*," [Online]. Available: <https://numpy.org/doc/>.
- [11] NLTK Project, "*nltk.translate.bleu_score and nltk.translate.meteor_score modules*," [Online]. Available: <https://www.nltk.org/api/nltk.translate.html>.
- [12] Intel Image Classification dataset (Kaggle). [Online]. Available: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>.
- [13] Flickr8k dataset (Kaggle / University of Illinois). [Online]. Available: <https://www.kaggle.com/datasets/adityajn105/flickr8k>.
- [14] Tim Asisten IF3270, "*Spesifikasi Tugas Besar 2: CNN & RNN/LSTM Image Captioning*," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026.

